Use LLMs to Build Repair Tools, Not Patch Generation: Cost-Effective Automated Program Repair

Anonymous submission

Abstract

Large language models (LLMs) repair buggy submissions to programming assignments well, but they are deployed *one query per submission*: every request re-transmits the same assignment description and test suite, so cost grows linearly with enrollment and recurs every term, while caching and cheaper models shrink only the constant, not the structure. Observing that assignment repair is a *many-instances, one-specification* regime, we propose CoEP, a framework that shifts LLM investment from the submission to the assignment. An LLM agent reads the specification, tests, and a few sample programs once, and synthesizes an executable symbolic repairer—problem-specific AST rewrite rules over a generic bounded edit search—whose every patch is independently test-validated and size-capped. Refined through validation feedback and then deployed locally, the repairer fixes subsequent submissions at zero marginal LLM cost, escalating the rest to conventional per-submission repair so that coverage is never sacrificed; it is also a transparent artifact that instructors can audit, produces minimal patches by construction, and repairs locally in under a second. Experiments on real student programs, against per-submission LLM repair, a symbolic non-LLM tool, and cost-reduction alternatives such as caching and model cascades, show that amortizing LLM reasoning into a verifiable per-assignment artifact preserves repair quality while fundamentally improving the cost structure of serving many instances of one specification.

Introduction

Timely, individualized feedback is the scarcest resource in programming education: introductory courses enroll hundreds of students on campus and tens of thousands online, each producing buggy submissions no instructor can examine one by one. Automated program repair (APR) has therefore long been proposed as a feedback engine for programming assignments: given a buggy submission and the assignment's test suite, produce a small patch that makes it correct while preserving the student's own approach (Yi et al. 2017; Gulwani, Radicek, and Zuleger 2018; Wang, Singh, and Su 2018; Hu et al. 2019; Ahmed et al. 2022).

This is an anonymized submission for review purposes only. Distribution, citation, or public sharing of this manuscript is strictly prohibited. Copyright and publication details will appear in the final version if accepted.

Large language models (LLMs) have made this vision practical. LLM-based repair now fixes the large majority of real student programs—up to 96.5% of buggy Python submissions in PyDex (Zhang et al. 2024)—at modest per-bug prices (Xia and Zhang 2024, 2022; Liu et al. 2024; Yang et al. 2024). But these systems all share a deployment paradigm that we argue is mismatched with the economics of the classroom: *every buggy submission triggers a fresh LLM query* that re-transmits the assignment description, the test suite, and formatting instructions before the model ever sees the student's code. On the real student corpus we study, 77% of the input tokens of a per-submission repair prompt are byte-identical across all submissions to the same assignment (55% in a contiguous cacheable prefix); the student's code is 23% of what the course pays to process, query after query, semester after semester. Provider-side prompt caching (Anthropic 2025; OpenAI 2024) discounts redundant prefixes but does not change the paradigm—here it cannot even engage, since the entire 439-token prompt sits below the provider's 1,024-token cacheable minimum—and every submission still requires a frontier-model round trip, with the attendant marginal cost, latency, rate limits, and external dependence.

Our key observation is that assignment repair is a *many-instances, one-specification* regime: a single assignment is attempted by hundreds of students whose failures cluster into recurring misconceptions (Gulwani, Radicek, and Zuleger 2018; Hu et al. 2019). Pre-LLM systems exploited exactly this structure (CLARA clustered correct solutions; Refactory refactored them into a per-problem search space), achieving cheap per-submission repair at the cost of hand-built machinery or large pools of prior solutions; per-submission LLM repair inverts the trade-off, automatic and general but paying for the same specification-level reasoning on every query.

We propose to get both halves: **spend the LLM once per assignment, not once per submission**. In CoEP (Cost-Effective Program repair), an LLM agent receives the assignment specification, test suite, a few reference solutions, and a handful of historical buggy submissions, and *writes a program repairer*: a deterministic Python module combining problem-specific rewrite rules with a generic edit-search library, validating every candidate against the visible tests under a patch-size cap. After at most three harness-driven refinement rounds it is frozen and deployed: submissions

are repaired *locally*, in under a second on average, at zero marginal LLM cost, and any the repairer cannot fix escalate to the conventional LLM path—so the hybrid never does worse than the paradigm it replaces.

On 5 real assignments from the public Refactory dataset (Hu et al. 2019)—1,685 buggy Python submissions written by university CS1 students—holding the underlying LLM fixed across all conditions, CoEP's synthesized repairers alone fix half of held-out submissions at zero marginal LLM cost (50.7% [48.3, 53.1] on the full corpus), and the central result is one of *inheritance*: fronting any per-submission protocol with CoEP preserves that protocol's quality set—verified instance by instance—while cutting its marginal cost 1.8–1.9×. Against the deployed plain-prompt convention the hybrid additionally repairs strictly more within the patch-size cap (90.0% vs. 86.0%; 8 unique wins, none reversed, concentrated in 3 of 5 assignments—significant at the submission level, p=0.008, suggestive at the cluster level), because unconstrained generations bust the cap on 14% of instances. The one-off synthesis cost of $0.256 per assignment is recouped after ≈197 repaired submissions on corpus-average economics—below the 337 buggy submissions per assignment this corpus averages in a *single* offering—with per-assignment break-evens of 38–704 disclosed in RQ2. The alternatives this observation invites do not close the gap: PyDex-style few-shot prompting raises cost 2.4× without raising quality (85.5%); the non-LLM Refactory tool, given *every* correct student solution, passes tests on 90.5% but busts the cap on a third of instances (56.0% quality); a cheap-model cascade floors at 88.5% for 1.8× CoEP's marginal cost; and granting the baseline CoEP's own enforcement—a size validator with one feedback retry—reaches 98.5%, but at $3.26×10^{-3}$ per submission. CoEP composes with all of them: fronting the cascade delivers 90.5% at the lowest marginal cost of any arm above 86% quality, and fronting the enforced-retry protocol repairs the *identical* 98.5% at 1.8× lower cost—for every fallback, the same quality at a smaller slope.

This paper makes the following contributions:

- **A new deployment paradigm for assignment repair.** We identify the many-instances, one-specification structure of programming-assignment APR and formulate *per-assignment repairer synthesis*: amortizing LLM reasoning into a reusable, locally executable repair artifact, with a cost model and break-even analysis for when it pays off.
- **The CoEP system.** A builder agent that synthesizes per-assignment repairers through a generate–validate–refine loop (Madaan et al. 2023; Shinn et al. 2023) over a repair-operator library; repairers are independently re-validated (test pass + patch-size cap) so they can never self-report success, and are backed by hybrid LLM escalation.
- **A same-model evaluation on real student programs.** On 5 assignments and 1,685 real buggy submissions from the public Refactory dataset, we compare both paradigms driven by the *same* frontier model against eight baselines spanning per-submission LLM repair (plain, cached, few-shot, enforced-retry), cost-reduction alternatives (cheap model, cascade, generic-search-only), and the non-LLM Refactory tool—under one validation harness, uniform cost accounting, and statistical tests; plus ablations isolating synthesized rules, refinement rounds, and builder capability.
- **Artifacts.** We release the harness, dataset-import pipeline, prompts, synthesized repairers, and all raw model outputs.[1]

The Cost Anatomy of Per-Submission Repair

To make the redundancy argument concrete, we decompose the per-submission repair prompt used by our baseline (and, mutatis mutandis, by PyDex- and ChatRepair-style systems (Zhang et al. 2024; Xia and Zhang 2024)). Averaged over the Refactory corpus, a single repair query consumes 439 input and 94 output tokens: a task preamble with the assignment description and visible tests (241 tokens, 55%), the buggy-submission section (99 tokens, 23%—the *only* part that varies between submissions to the same assignment), and fixed task instructions (99 tokens, 23%). 77% of every query is thus byte-identical across a class, and only the contiguous 55% prefix ahead of the code is even cacheable; richer prompt styles widen the gap (the PyDex-style few-shot variant re-transmits 1,493 identical tokens per query, 88% of its input).

Two consequences follow. (1) *Caching is a discount, not a fix—and here, not even a discount.* Prompt caching bills a shared prefix at 0.1× after a 1.25× write (Anthropic 2025); with no minimum it would cut per-submission cost 23%, but CS1 repair prompts (439 tokens) fall below the provider's 1,024-token cacheable minimum, so even that bound is unreachable; caching does engage on the bloated few-shot prompt (1,705 tokens; $6.5 to $2.9×10^{-3}$)—the redundancy is real, but provider caching serves it at the wrong granularity. (2) *The redundancy is exactly the amortizable part.* What is constant per assignment—understanding the specification, the tests, and the space of likely mistakes—can be reified *once* into an executable artifact. That is the move CoEP makes, and the rest of the paper quantifies what it buys.

Approach

Problem Formulation

An *assignment* is a triple $A = (d, T, R)$: a natural-language description d, a visible test suite $T = \{(x_i, y_i)\}$ over a designated entry function (individual tests may call named auxiliary functions the assignment requires), and a small set R of reference solutions. A *repair* of a buggy submission b is a program p such that (i) p passes all of T and (ii) $\mathrm{RPS}(b, p) \le \tau$, where RPS is the token-level relative patch size—the Levenshtein distance between the lexical token sequences of b and p, normalized by $|b|$—following the patch-size metrics of Refactory and PyDex (Hu et al. 2019; Zhang et al. 2024). The cap τ (we use $\tau = 0.35$) operationalizes the pedagogical requirement that a repair be feedback on the *student's* program: it rules out replacing the submission with a reference solution, which passes tests but teaches nothing (Yi et al. 2017).

[1] Anonymized repository submitted as supplementary material.

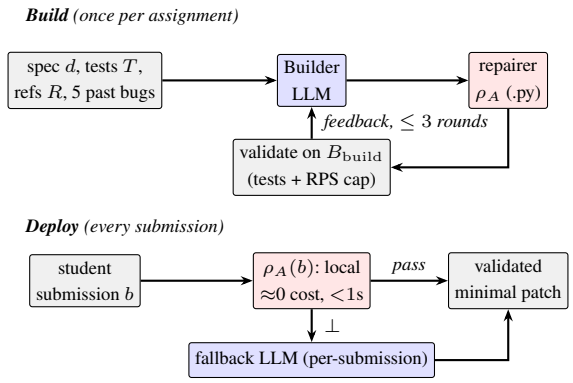


Figure 1: CoEP. LLM effort (blue) is spent once per assignment to synthesize a local repairer (red); at deployment, half of submissions never touch a model.

Conventional LLM-based APR maps each b to a query $\text{LLM}(d, T, b)$. CoEP instead synthesizes, once per assignment, a *repairer* $\rho_A : b \mapsto p \cup \{\perp\}$—an ordinary Python module with no model access—and deploys the hybrid $\text{repair}(b) = \rho_A(b)$ if $\rho_A(b) \neq \perp$ (at $\approx$0 cost), else $\text{LLM}(d, T, b)$.

Repairer Synthesis

A builder agent receives d, T, R, and k=5 historical buggy submissions B_{build}, together with the documentation of a small *repair-operator library* that we provide as the substrate for all synthesized repairers. The library contributes three reusable capabilities:

(1) Generic edit search. `candidates_generic` enumerates candidate patches by applying small semantic edits at every applicable AST site—comparison/arithmetic/Boolean operator swaps, integer off-by-one shifts, Boolean-constant flips, index shifts, and argument swaps—singly or in pairs. This is the inversion of the classical mutation-operator space (Just et al. 2014) and plays the role that hand-curated fix templates play in template-based APR (Kim et al. 2013; Liu et al. 2019).

(2) Test-driven validation. `first_passing` runs candidates against the full suite T in a sandbox and returns the first that passes *all* tests within the RPS cap; synthesized code is contractually forbidden from returning an unvalidated patch.

(3) Failure diagnosis. `failing_tests` exposes the input program's failing-test pattern, letting repairers dispatch (e.g., output sorted ascending rather than descending selects order-reversal rules first).

The builder's essential creative work is the *problem-specific rule set*: AST rewrite rules that anticipate the misconceptions students exhibit for *this* assignment—occurrence thresholds off by an idiom in `unique_day` (`>=2` for `!=1`), a tuple accumulator discarding non-maxima in `sort_age`, a helper copied from the previous exercise in `top_k`—derived from d, T, R, and B_{build}. These rules generate edits far outside the generic operator space, and our ablation shows they, not the generic search, carry the paradigm (RQ3).

Algorithm 1 Per-assignment repairer synthesis

Require: assignment (d, T, R), sample bugs B_{build}, rounds $R_{\max}$

$\rho \leftarrow \text{LLM}_{\text{build}}(d, T, R, B_{\text{build}}, \text{lib})$ ▷ single shot
for $r = 2 \ldots R_{\max}$ **do**
  $F \leftarrow \{(b, \text{outcome}(\rho, b)) : b \in B_{\text{build}}\}$ ▷ harness
  if all outcomes are REPAIRED **then**
    return ρ
  end if
  $\rho \leftarrow \text{LLM}_{\text{refine}}(\rho, F)$ ▷ single shot
end for
return ρ

Generate–validate–refine. Because a single-shot synthesized module can contain its own defects, the harness closes the loop (Algorithm 1): it installs the module, applies it to B_{build}, *independently* re-validates every returned patch (sandboxed test execution plus RPS check—the repairer's own claims are never trusted), and, if any sample bug is unrepaired, sends the module back with per-bug outcomes and the failing submissions for revision, up to $R_{\max}$=3 total rounds. This is a deliberately austere instantiation of self-refinement (Madaan et al. 2023; Shinn et al. 2023): every LLM interaction is a single prompt–response pair, so the entire build cost is the token count of at most $2R_{\max}$ message payloads, measured directly from the raw strings.

Deployment. The frozen repairer runs in a subprocess with a wall-clock budget (60s; observed mean 0.8s) and no network access; its patches are re-validated by the grading harness before reaching a student, so a defective repairer degrades to $\perp$ (escalation), never to a wrong "repair", and escalations follow the conventional per-submission path with their cost counted against CoEP in every comparison. (Caching past *patches* would amortize too, but a patch is specific to one buggy program; a synthesized rule generalizes over a misconception class.)

Experimental Setup

We ask three research questions: **RQ1 (Effectiveness & quality).** How do CoEP's synthesized repairers compare to per-submission frontier-LLM repair in repair rate and patch quality? **RQ2 (Cost).** What are the token/dollar costs of both paradigms, when does repairer synthesis break even, and what is the asymptotic saving? **RQ3 (Sources of power).** How much do the synthesized problem-specific rules and the refinement rounds contribute?

Dataset

We evaluate on the Refactory dataset (Hu et al. 2019): real submissions by 361 university students to five assignments of a CS1 Python course at the National University of Singapore—the standard real-student corpus of the assignment-repair literature, used by Refactory, Verifix, PyDex, and AAAI-25's counterexample-guided repair (Ahmed et al. 2022; Zhang et al. 2024; Orvalho, Janota, and Manquinho 2025). The assignments (`search`, `unique_day`/`unique_month`/

`contains_unique_day`, `remove_extras`, `sort_age`, `top_k`) span search, multi-function date logic, deduplication, sorting, and selection; one requires three separate functions, which our harness supports through per-test entry points. We import each assignment's handout, its instructor test suite (5–11 input/output tests per assignment), and 3 reference solutions sampled from the instructor-provided reference plus correct student submissions (seed 7). Of the corpus's 1,783 buggy submissions, 1,685 compile and fail at least one parsed test after import; from these, per assignment, 5 sampled submissions form the builder's B_{build} and 40 disjoint sampled submissions are held out for evaluation (n=200; seed 7, fixed before any experiment)—a size that keeps eight experimental arms tractable at ±5–7pp Wilson intervals.

This corpus has circulated publicly since 2019 and is plausibly in the training data of every current LLM; we use it *deliberately*. Memorization, if present, inflates every LLM-driven arm symmetrically—the per-submission baselines and CoEP's builder are the *same model*—so paradigm comparisons remain valid, while the non-LLM tool and our generic search are unaffected; what memorization cannot do is pass our validation harness on the builder's behalf, since every patch is re-executed against the instructor tests. We analyze the residual asymmetries in Discussion.

Baselines

Per-submission LLM repair is the deployment paradigm of PyDex/MMAPR, ChatRepair, and FastFixer (Zhang et al. 2024, 2022; Xia and Zhang 2024; Liu et al. 2024): one query per submission containing the handout, the visible tests, the buggy program, and minimal-patch instructions. We instantiate it with the *same* frontier model that powers CoEP's builder, so every observed difference is attributable to the paradigm, not the model. **PyDex-style few-shot repair** enriches each query with the two ingredients PyDex found most effective (Zhang et al. 2024): five real (buggy, fixed-by-the-same-student) example pairs from the *same* assignment—drawn from B_{build} students where their own later fix passes the suite, never from evaluation submissions—plus the submission's concrete failing-test feedback. This gives the per-submission paradigm the same assignment-specific information budget CoEP's builder receives. **Cached per-submission repair** re-prices the plain queries under provider prompt caching (shared prefix written once at $1.25\times$, read at $0.1\times$) (Anthropic 2025). **Cheap-model repair and cascade**: the same per-submission queries executed by a Haiku-tier model ($1/$5 per Mtok), alone and in a FrugalGPT-style cascade (Chen, Zaharia, and Zou 2024) that escalates its quality failures to the frontier model—the strongest *zero-build-cost* competitors to amortization. **Generic-hybrid**: our generic edit search run with no LLM synthesis at all, backed by the same frontier fallback—the strongest free local competitor, and an ablation of the synthesized rules. **Refactory** (Hu et al. 2019), the classical non-LLM per-assignment tool, run in its strongest published configuration (online refactoring, structure mutation, block repair, 100% sampling of correct solutions): for each assignment it receives *all* correct student submissions (2,442 across the five assignments) plus the reference—orders of magnitude more per-assignment data than CoEP's 3 references and 5 sample bugs—and its output programs are re-validated under our harness and metrics.

Metrics

Test-pass rate: fraction of evaluation submissions for which the system produces a program passing all visible tests (Hu et al. 2019; Zhang et al. 2024). **Quality repair rate** (primary): test-pass *and* $\text{RPS} \leq 0.35$—the constraint CoEP enforces by construction, applied identically to every arm—since a test-passing rewrite that discards the student's approach is not usable feedback (Yi et al. 2017). **Relative patch size (RPS)**: token-level edit distance normalized by program length (Hu et al. 2019; Zhang et al. 2024); smaller is pedagogically better. **Cost**: token counts of every raw prompt/response string under one fixed convention, priced at a representative frontier tier ($3/$15 per Mtok in/out; Claude Sonnet standard pricing, July 2026) with cache pricing as above, reported as tokens, USD per submission, and USD per assignment, following the per-bug cost accounting of ChatRepair (Xia and Zhang 2024). Repair rates carry 95% Wilson intervals; paired comparisons use exact McNemar tests at the submission level, and because submissions cluster within five assignments, we report per-assignment directions alongside every pooled test.

Protocol

All LLM interactions—per-submission repairs in every LLM arm and every builder/refinement call—are single-shot prompt→response exchanges executed by the stated model (Claude Sonnet or Haiku tier, July 2026 snapshots) under identical instructions, with no code execution or test access at generation time; all validation is performed by our harness. Local search is deterministic; LLM calls are single-sample (n=1), matching the deployment setting of one repair attempt per submission. Costs are computed from the exact message strings with a fixed tokenizer approximation applied identically to all arms (chars/3.6; provider tokenizers are unavailable offline); because the arms differ in input/output mix, we release every raw string for exact re-tokenization and note that a ±20% differential calibration error shifts break-even points by at most the same factor. To probe sensitivity to builder capability, we additionally re-ran the entire synthesis pipeline with the Haiku-tier model as the builder (same prompts, same $R_{\max}$). The Refactory tool ran in its original configuration ported to Python 3.11, on the same evaluation submissions. Synthesized repairers used a 60s timeout and τ=0.35 throughout; experiments ran on CPU only.

Results

RQ1: Effectiveness and Patch Quality

Table 1 presents the headline comparison. Every LLM arm passes the visible tests on essentially all submissions—on CS1-scale programs, functional repair is saturated (Zhang et al. 2024)—but *quality* repair is not: 14% of the frontier baseline's test-passing patches exceed the size cap, rewriting rather than repairing the student's program (Yi et al. 2017). PyDex-style enrichment with real same-assignment example

	Test-pass rate	Quality (RPS≤.35)	mean RPS	sec/ sub
Frontier per-subm.	**100.0**	86.0 [80.5, 90.1]	0.102	—
PyDex-style few-shot	**100.0**	85.5 [80.0, 89.7]	0.108	—
Cheap-model (Haiku)	88.0	65.0 [58.2, 71.3]	0.118	—
Refactory (non-LLM)	90.5	56.0 [49.1, 62.7]	0.187	5.5
Generic-only	6.5	6.5 [3.8, 10.8]	0.023	5.6
CoEP-local	47.5	47.5 [40.7, 54.4]	0.127	0.8
CoEP+frontier fb.	**100.0**	90.0 [85.1, 93.4]	0.120	—
CoEP+cascade fb.	**100.0**	**90.5** [85.6, 93.8]	0.129	—

Table 1: Effectiveness on 200 held-out real submissions (95% Wilson CIs). Quality repair rate (test-pass within the shared patch-size cap) is primary; mean RPS is over cap-compliant repairs; "sec/sub" is local CPU time.

fixes and execution feedback does not help (85.5%, $2.4\times$ the cost): the failure is not a knowledge gap that more context fills, but the absence of an enforced size discipline. The classical Refactory tool, despite receiving every correct student solution, drops to 56.0% under the same discipline (its patches average 0.386 RPS across test-passing repairs, twice our cap), quantifying on shared data the invasive-fix concern about symbolic tools (Orvalho, Janota, and Manquinho 2025).

CoEP's synthesized repairers alone fix 47.5% at zero marginal LLM cost (mean 0.8 s per submission on one CPU core), with patches comfortably inside the cap (mean RPS 0.127; on the 87 instances both repair, the frontier's accepted patches are slightly smaller, 0.093 vs. 0.115), and they are *safe*: across 200 attempts, zero test-failing and zero over-cap patches were emitted—the only failure mode is abstention ($\perp$), never a wrong repair. The local path's advantage is therefore not average patch size but *enforcement*: escalating abstentions to the frontier model lifts the hybrid to 90.0%, above the per-submission baseline itself (86.0%), because on 8 submissions the size-capped local search finds a minimal repair where the baseline's one-shot generation overshoots the cap (exact McNemar, discordant 0:8, p=0.008; the ordering is never reversed in any assignment; strictly better in three, tied in two). The tool comparison is no artifact of re-validation: Refactory's own oracle reports 91.0% on these runs, within a point of our 90.5% test-pass figure—the gap under our *quality* metric is entirely its patch sizes. Guarantees, not just averages, change character: a prompt can *request* minimality, but only a validator can *enforce* it—a point we return to with an enforcement-parity baseline in RQ2.

Scaling the zero-marginal local arm to the *full* corpus (all 1,660 non-build submissions) confirms the estimate tightly: 50.7% coverage [48.3, 53.1], per-assignment 27.9–71.4%. The dispersion tracks idiom diversity (`search` is variations of one loop; `remove_extras` attracts diverse list-surgery styles whose aliasing bugs defeat rules from five samples); the hybrid absorbs it, and build-round trajectories signal it *before* deployment—a routing cue for larger build budgets on harder assignments.

	Quality RR	Build $	$/sub (m$)
Frontier per-subm.	86.0	0	2.73
+ caching (no-min. bound)	86.0	0	2.10
+ validator, 1 retry	98.5	0	3.26
PyDex-style few-shot	85.5	0	6.53
+ prompt caching	85.5	0	2.91
Cheap-model (Haiku)	65.0	0	0.90
+ validator, 1 retry	81.5	0	1.32
Cascade (cheap→front.)	88.5	0	1.94
w/ cheap self-retry	89.5	0	1.90
Generic-hybrid	86.0	0	2.55
Refactory-tool + frontier fb.	86.5	0	1.33
CoEP + cascade fb.	90.5	0.256	**1.07**
CoEP + self-retry casc.	91.0	0.256	1.08
CoEP + frontier fb.	90.0	0.256	1.43
CoEP + retry fb.	**98.5**	0.256	1.78

Table 2: The amortization ladder (m$ $= 10^{-3}$ USD). "+ validator, 1 retry" gives the baseline CoEP's enforcement; the plain-prompt caching row is a no-minimum bound unavailable in practice. Fronting *any* protocol with CoEP preserves its quality at lower marginal cost.

RQ1. Synthesized repairers fix 47.5% locally with zero unsafe outputs; the hybrid's quality repair rate (90.0–90.5%) *exceeds* every deployed single-shot configuration (p=0.008; never reversed in any assignment, strictly better in three) and clears the non-LLM tool by 34 points; enforcement itself generalizes—RQ2's validated-retry protocol and its CoEP composition.

RQ2: Cost and Break-Even

Table 2 prices every configuration. Within the pay-per-query paradigm the trade-off is stark: the cheap model saves $3\times$ but surrenders 21 quality points; caching is inapplicable to the plain prompt (the table's row is the no-minimum bound) and brings the few-shot variant only to $\$2.91\times10^{-3}$; and enforcement—a validator plus one minimality-feedback retry, the PyDex-style iterative protocol—lifts the frontier baseline to 98.5% for $\$3.26\times10^{-3}$, the cheap model to 81.5% for \$1.32, and the cascade (cheap self-retry before escalation) to 89.5% for \$1.90: every zero-build configuration, enforced or not, stays below CoEP+cascade on quality, cost, or both. The classical tool can also front the fallback: Refactory-then-frontier reaches 86.5% at $\$1.33\times10^{-3}$ with zero build cost—but CoEP+cascade dominates it on *both* axes (90.5% at \$1.07), CoEP+frontier repairs strictly more (discordant 7:0, p=0.016), and the tool needs all 2,442 correct solutions and 5.5 s per submission where CoEP needs 3 references, 5 bugs, and 0.9 s. The retry protocol defines this study's quality ceiling—and CoEP *inherits* it: since the local path never emits an unsafe patch, fronting any protocol preserves its quality set while diverting half its traffic. Empirically the inheritance is exact: CoEP+retry repairs the *identical* 197/200 submissions as the retry arm (discordant 0:0, per-assignment identical) at $1.8\times$ lower marginal cost, while CoEP+frontier beats the plain baseline outright (p=0.008) and CoEP+cascade does so at the lowest marginal

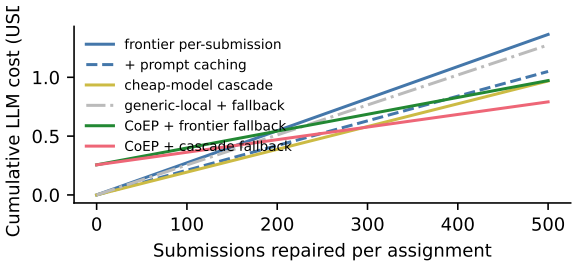


Figure 2: Cumulative LLM cost per assignment on real submission volumes. Pay-per-query optimizations (caching, cascade) change the slope; only amortization moves cost into a one-off intercept, and the composed CoEP+cascade configuration dominates the cascade beyond ≈295 submissions while also repairing more.

cost of any quality-competitive arm (p=0.004; vs. the cascade alone, discordant 4:0, n.s.; only the 65%-quality Haiku arm is cheaper). Hybrid cost stays linear in enrollment—the claim is a 1.8–2.5× smaller slope plus an intercept, not zero marginal cost—and the slope shrinks as coverage grows.

Synthesizing an assignment's repairer costs $0.256 on average (51227 prompt + 75015 response tokens across five assignments and ten builder calls; the price of ≈94 baseline repairs, paid once). Let C be the build cost and Δ the marginal saving per submission over a competitor; synthesis wins once $N > N^* = C/\Delta$. Against the frontier baseline $N^* \approx 197$ (≈154 with cascade fallback); against the cascade itself ≈295; at matched 98.5% quality, CoEP+retry recoups its build within ≈173. Because build cost and coverage vary by assignment, so does N^*: computed per assignment with full-corpus coverage, `search`/`unique_day`/`remove_extras` break even at 38/88/241 submissions—inside their own single-offering volumes (557/417/306)—while `sort_age` needs ≈1.2 offerings (368 vs. 297) and `top_k` (N^*≈704 vs. 108) pays off only at multi-offering or MOOC scale: amortization is per-assignment economics, and the build decision can be made per assignment from the build trajectory alone. Composite-arm dollars are measured per-instance sums, not products of averages; escalated subsets cost more than average because escalation adversely selects harder, longer submissions (the frontier repairs 67% of Haiku-escalated instances against 86% overall), which is why, e.g., the cascade's $1.94 exceeds the naive $0.90 + 0.35×2.73 = $1.86. Figure 2 plots the full ladder.

Beyond cost, half of submissions are repaired in under a second on a CPU with no network dependency, rate limits, or nondeterminism—properties hosted per-query APIs do not offer, and that matter most at deadline-night spikes.

RQ2. Fronting any per-submission protocol with CoEP preserves its quality at a 1.8–2.5× smaller cost slope (98.5% at $1.78 vs. $3.26×10^{-3}; 90.5% at $1.07×10^{-3}); the $0.256 build breaks even in ≈154–295 submissions, inside one offering for four of five assignments.

RQ3: Where Does the Power Come From?

Synthesized rules vs. generic search. The generic-only ablation—the same operator library and validation, with no LLM-synthesized rules—repairs 6.5% of real student bugs (13/200), against CoEP's 47.5% (exact McNemar, discordant 0:82, $p<10^{-24}$). The collapse is diagnostic: real student bugs are almost never single-operator slips but misspelled names, thresholds off by an idiom (`>=2` for `!=1`), helpers copied from the previous exercise, wrong-type accumulators—structural rewrites no generic operator produces. On this corpus, *synthesized understanding is not an increment over search; it is 86% of everything the local path repairs*.

Refinement rounds. Two of five frontier builders passed all build samples in one shot; feedback rounds repaired the other three ($15/25 \rightarrow 25/25$ build success), each from a *delivery* defect, not a comprehension one: `unique_day`'s round-1 rules were semantically right but rendered patches via `ast.unparse`, whose reformatting blew the cap on every candidate ($0/5 \rightarrow 5/5$ after switching to surgical line edits); `sort_age` twice produced correct-but-oversized rewrites before a 14-token insertion; `top_k` never survived validation until round 3 emitted per-pattern corrected programs ($1/5 \rightarrow 5/5$). Validation surfaces these defects; feedback fixes them—single-shot synthesis would have shipped three broken repairers of five.

Builder capability. Re-running the entire synthesis pipeline with the Haiku-tier model as builder (same prompts, same $R_{\max}$=3) yields repairers that fix 10.0% of held-out submissions (20/200), barely above the generic floor: only one of five cheap builders ever passed all build samples ($14/25$ final build success vs. $25/25$), two never repaired a single one, and their failure modes—whole-file reformatting, unvalidated candidates—are exactly the delivery defects the frontier builder learns to fix from feedback. Its rules cost $0.022 per assignment but repair too little to matter; upgrading to the frontier builder pays for itself within ≈230 submissions—synthesis is a frontier-capability task, but that capability is bought once per assignment, not once per submission; that is the entire point.

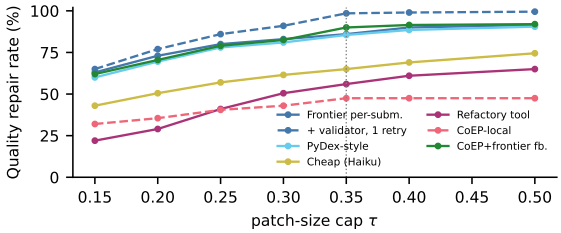


Figure 3: Quality repair rate vs. the patch-size cap τ, recomputed from stored patch sizes. Orderings are stable; CoEP curves are conservative lower bounds for $\tau<0.35$ (frozen repairers searched under 0.35).

RQ3. Generic edit search fixes 6.5% of real bugs—synthesized rules are 86% of what the local path repairs ($p<10^{-24}$); refinement rescues three of five builders ($15/25 \rightarrow 25/25$); a cheap-model builder collapses to 10.0% vs. the frontier's 47.5%.

Discussion and Limitations

Construct validity. Repair rate is measured against the instructor's visible tests, as in prior assignment-repair work (Hu et al. 2019; Zhang et al. 2024); a patch could overfit the suite, and CoEP's search optimizes that oracle directly, but the RPS cap bounds the degenerate form of that overfit (wholesale replacement)—and it is the *per-submission* arms that exceed it in practice (14% of frontier patches, a third of the tool's) while the local path produced zero unsafe outputs. The threshold (τ=0.35) follows Refactory and PyDex, and a τ-sweep from stored patch sizes (Figure 3) shows the orderings are not its artifact: the validated-retry arm dominates at every τ, the hybrid tracks or exceeds the plain baseline for $\tau \geq 0.25$ (below that the recomputation is conservative—frozen repairers searched under 0.35), and the tool's collapse appears at every threshold. Because CoEP's search optimizes the visible suite directly, suite overfitting favors it differentially; a hidden-oracle audit (60 seeded random inputs per assignment, differential against the instructor reference) bounds the effect: 92.6% of accepted local patches agree with the reference on every probe vs. 97.1% of the baseline's—a measured 4.5pp differential, and a probe the deployment can add as a third validator at zero LLM cost. The cost model prices one tier from modeled (not billed) token counts; released raw strings let any pricing and tokenizer substitute into RQ2's closed-form break-even.

Internal validity. Both paradigms use the same model and the same independent validation harness; the builder sees 5 sample bugs and never the 40 evaluation submissions. Hybrid-vs-baseline McNemar tests are structurally one-sided (the hybrid contains the fallback) and submissions cluster in five assignments, where a cluster-level sign test on the 8 unique wins (3 wins, 0 losses, 2 ties) is not significant—the quality edge over the plain baseline is consistent but suggestive; the load-bearing comparisons are the symmetric ones we also report: the enforcement-parity arm (both sides validated, equal attempts; quality sets identical at lower cost) and patch sizes on shared instances. Reusing cached *patches* across near-duplicate submissions would cover at most 33% of evaluation instances (those with an identical normalized-AST twin outside the split), below the local rules' 50.7%—rules generalize where retrieval cannot. The corpus is public since 2019, so training-data contamination is possible—and, by design, *symmetric*: the same model powers every LLM arm, so memorization inflates the baseline at least as much as the builder, whose output must additionally survive execution against the instructor tests; corroboratingly, a builder reciting known fixes would not need refinement rounds to repair `ast.unparse` formatting defects, nor would the cheap builder—with the same putative memorized knowledge—collapse to 10%. Each assignment reflects one stochastic builder trajectory; the refinement loop absorbed all three observed failures, and multi-seed replication is mechanical with the released harness. LLM calls ran through an agent harness constrained to a stateless single-shot endpoint (no tool use, no test execution); costs are computed from the raw message payloads under one fixed approximate tokenizer, so uniform calibration error cancels in every ratio (a differential error shifts break-evens by at most its magnitude; Protocol), and we release all raw strings for exact re-tokenization.

External validity. We evaluate five assignments from one course, one language, and one model family (two tiers); multi-function projects beyond `unique_day`'s three functions are open. Evaluation subsamples 40 buggy submissions per assignment (seeded, fixed before any experiment) from the 1,685 available; Wilson intervals quantify the uncertainty, and the local arm's full-corpus re-run (RQ1) confirms the estimate. Local repair is heterogeneous (28–71% by assignment) and bounded by what five sample bugs can teach; richer build sets or clustering the full history (Gulwani, Radicek, and Zuleger 2018) are extensions the framework prices explicitly, and the economics assume assignments recur at scale—this corpus's own regime (108–557 buggy submissions per assignment per offering).

Related Work

APR for programming assignments. Pre-LLM systems exploit per-assignment structure—CLARA's clustering (Gulwani, Radicek, and Zuleger 2018), Sarfgen's aligned examples (Wang, Singh, and Su 2018), Refactory's refactoring (Hu et al. 2019), Verifix (Ahmed et al. 2022)—and Yi et al. (2017) established test-based repair rates and the feedback concern behind patch-size metrics; all need many prior solutions or hand-built machinery, and our head-to-head run shows the quality cost of invasive patches. At AAAI the setting stays active from the per-submission side: counterexample-guided and preference-tuned minimal repair at AAAI-25 (Orvalho, Janota, and Manquinho 2025; Dai et al. 2025), and AAAI-26's learner-tailored repair, which retrieves per-assignment example fixes to guide each query (Dai et al. 2026)—amortizing *data* into a retrieval index while every repair still pays for an LLM call (our PyDex-style arm bounds this design's economics). CoEP instead amortizes *reasoning* into an executable artifact and pays only on escalation.

LLM-based repair. AlphaRepair recast repair as generation (Xia and Zhang 2022); ChatRepair adds conversational test feedback and per-bug cost accounting (Xia and Zhang 2024); PyDex/MMAPR target student programs (Zhang et al. 2024, 2022); FastFixer accelerates per-query inference (Liu et al. 2024); CREF adds a tutor in the loop (Yang et al. 2024). All are per-submission; CoEP is complementary (any can serve as its fallback) but shifts the primary cost to the assignment.

Template- and pattern-based APR. PAR and TBar showed that a fixed library of human-curated fix templates repairs a surprising share of real bugs (Kim et al. 2013; Liu et al. 2019); CoEP's generic operator layer is a mutation-inversion instance of this idea (Just et al. 2014), and its novelty is the *synthesized, assignment-specific* template layer, authored per specification by an LLM rather than curated globally by humans.

Cost-efficient LLM inference. FrugalGPT's cascades route queries between models of different prices (Chen, Zaharia, and Zou 2024); SRepair engineers cheap open-model pipelines (Xiang et al. 2024); provider caching discounts repeated prefixes (Anthropic 2025; OpenAI 2024). All optimize *within* the pay-per-query paradigm and compose with CoEP, which instead removes queries: half of submissions never reach a model.

LLMs synthesizing reusable tools. Voyager's skill library and LLMs-as-tool-makers show agents writing reusable code artifacts amortized across future episodes (Wang et al. 2024; Cai et al. 2024); self-refinement improves one-shot generations (Madaan et al. 2023; Shinn et al. 2023). CoEP adds the twists education forces: the tool must be *verifiable* (never-trust-self-reports) and *quality-constrained* (RPS cap), because its output is feedback to a learner.

Conclusion

Programming-assignment repair has a structure per-submission querying ignores: one specification, many failures, recurring misconceptions. On real student programs, CoEP turns 26 cents of frontier-model reasoning into a permanent, local, verifiable repairer per assignment; the hybrid beats the deployed per-submission paradigm on quality repairs (90.5% vs. 86.0%) and matches the strongest enforced-retry protocol exactly (98.5%) at a 1.8–$2.5\times$ smaller cost slope, breaking even inside one offering. The recipe—*amortize LLM reasoning into verifiable executable artifacts wherever many instances share a specification*—extends to grading rubrics, hint generation, and beyond.

Ethical Statement

CoEP automates feedback, not grading, under instructor oversight: unhandled students get full-LLM treatment, local repair is auditable, and all data are public (Refactory).

References

Ahmed, U. Z.; Fan, Z.; Yi, J.; Al-Bataineh, O. I.; and Roychoudhury, A. 2022. Verifix: Verified Repair of Programming Assignments. *ACM Transactions on Software Engineering and Methodology*, 31(4).

Anthropic. 2025. Prompt Caching — Claude API Documentation. https://docs.anthropic.com/en/docs/build-with-claude/prompt-caching. Cache reads are billed at 0.1x the base input-token price; accessed July 2026.

Cai, T.; Wang, X.; Ma, T.; Chen, X.; and Zhou, D. 2024. Large Language Models as Tool Makers. In *The Twelfth International Conference on Learning Representations, ICLR 2024*.

Chen, L.; Zaharia, M.; and Zou, J. 2024. FrugalGPT: How to Use Large Language Models While Reducing Cost and Improving Performance. *Transactions on Machine Learning Research*.

Dai, Z.; Chen, B.; Zhao, Z.; Tang, X.; Wu, S.; Yao, C.; Gao, Z.; and Chen, J. 2025. Less is More: Adaptive Program Repair with Bug Localization and Preference Learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*.

Dai, Z.; Zhao, Z.; Wang, H.; Tang, X.; Wu, S.; Yao, C.; Gao, Z.; and Chen, J. 2026. Learner-Tailored Program Repair: A Solution Generator with Iterative Edit-Driven Retrieval Enhancement. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 30467–30475.

Gulwani, S.; Radicek, I.; and Zuleger, F. 2018. Automated Clustering and Program Repair for Introductory Programming Assignments. In *Proceedings of the 39th ACM SIGPLAN Conference on Programming Language Design and Implementation, PLDI 2018*, 465–480. ACM.

Hu, Y.; Ahmed, U. Z.; Mechtaev, S.; Leong, B.; and Roychoudhury, A. 2019. Re-Factoring Based Program Repair Applied to Programming Assignments. In *34th IEEE/ACM International Conference on Automated Software Engineering, ASE 2019*. IEEE.

Just, R.; Jalali, D.; Inozemtseva, L.; Ernst, M. D.; Holmes, R.; and Fraser, G. 2014. Are Mutants a Valid Substitute for Real Faults in Software Testing? In *Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering, FSE 2014*. ACM.

Kim, D.; Nam, J.; Song, J.; and Kim, S. 2013. Automatic Patch Generation Learned from Human-Written Patches. In *Proceedings of the 35th International Conference on Software Engineering, ICSE 2013*. IEEE.

Liu, F.; Liu, Z.; Zhao, Q.; Jiang, J.; Zhang, L.; Li, G.; Sun, Z.; Li, Z.; and Ma, Y. 2024. FastFixer: An Efficient and Effective Approach for Repairing Programming Assignments. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering, ASE 2024*. ACM.

Liu, K.; Koyuncu, A.; Kim, D.; and Bissyandé, T. F. 2019. TBar: Revisiting Template-Based Automated Program Repair. In *Proceedings of the 28th ACM SIGSOFT International Symposium on Software Testing and Analysis, ISSTA 2019*, 31–42. ACM.

Madaan, A.; Tandon, N.; Gupta, P.; Hallinan, S.; Gao, L.; Wiegreffe, S.; Alon, U.; Dziri, N.; Prabhumoye, S.; Yang, Y.; Gupta, S.; Majumder, B. P.; Hermann, K.; Welleck, S.; Yazdanbakhsh, A.; and Clark, P. 2023. Self-Refine: Iterative Refinement with Self-Feedback. In *Advances in Neural Information Processing Systems 36, NeurIPS 2023*.

OpenAI. 2024. Prompt Caching — OpenAI API Documentation. https://platform.openai.com/docs/guides/prompt-caching. Automatic prefix caching with discounted cached input tokens; see also https://openai.com/index/api-prompt-caching/; accessed July 2026.

Orvalho, P.; Janota, M.; and Manquinho, V. 2025. Counterexample Guided Program Repair Using Zero-Shot Learning and MaxSAT-based Fault Localization. In *Proceedings of the AAAI Conference on Artificial Intelligence*.

Shinn, N.; Cassano, F.; Gopinath, A.; Narasimhan, K.; and Yao, S. 2023. Reflexion: Language Agents with Verbal Reinforcement Learning. In *Advances in Neural Information Processing Systems 36, NeurIPS 2023*.

Wang, G.; Xie, Y.; Jiang, Y.; Mandlekar, A.; Xiao, C.; Zhu, Y.; Fan, L.; and Anandkumar, A. 2024. Voyager:

An Open-Ended Embodied Agent with Large Language Models. *Transactions on Machine Learning Research*. ArXiv:2305.16291.

Wang, K.; Singh, R.; and Su, Z. 2018. Search, Align, and Repair: Data-Driven Feedback Generation for Introductory Programming Exercises. In *Proceedings of the 39th ACM SIGPLAN Conference on Programming Language Design and Implementation, PLDI 2018*. ACM.

Xia, C. S.; and Zhang, L. 2022. Less Training, More Repairing Please: Revisiting Automated Program Repair via Zero-Shot Learning. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, ESEC/FSE 2022*. ACM.

Xia, C. S.; and Zhang, L. 2024. Automated Program Repair via Conversation: Fixing 162 out of 337 Bugs for $0.42 Each using ChatGPT. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis, ISSTA 2024*, 819–831. ACM.

Xiang, J.; Xu, X.; Kong, F.; Wu, M.; Zhang, Z.; Zhang, H.; and Zhang, Y. 2024. How Far Can We Go with Practical Function-Level Program Repair? arXiv preprint arXiv:2404.12833.

Yang, B.; Tian, H.; Pian, W.; Yu, H.; Wang, H.; Klein, J.; Bissyandé, T. F.; and Jin, S. 2024. CREF: An LLM-Based Conversational Software Repair Framework for Programming Tutors. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis, ISSTA 2024*, 882–894. ACM.

Yi, J.; Ahmed, U. Z.; Karkare, A.; Tan, S. H.; and Roychoudhury, A. 2017. A Feasibility Study of Using Automated Program Repair for Introductory Programming Assignments. In *Proceedings of the 11th Joint Meeting on Foundations of Software Engineering, ESEC/FSE 2017*, 740–751. ACM.

Zhang, J.; Cambronero, J.; Gulwani, S.; Le, V.; Piskac, R.; Soares, G.; and Verbruggen, G. 2022. Repairing Bugs in Python Assignments Using Large Language Models. arXiv preprint arXiv:2209.14876. Preprint; the archival version of this work appeared as PyDex at OOPSLA 2024 (Zhang et al. 2024).

Zhang, J.; Cambronero, J. P.; Gulwani, S.; Le, V.; Piskac, R.; Soares, G.; and Verbruggen, G. 2024. PyDex: Repairing Bugs in Introductory Python Assignments using LLMs. *Proceedings of the ACM on Programming Languages*, 8(OOPSLA1).

Reproducibility Checklist

General paper structure. Conceptual outline of the approach: yes (Algorithm 1, Figure 1). Claims vs. speculation delineated: yes (RQ answer boxes state measured claims; the closing recipe is marked as argument). Pedagogical references for background: yes (Related Work).

Theoretical contributions. This paper makes no formal theoretical claims; the break-even expression in RQ2 is elementary algebra over measured quantities, with all symbols defined in the text.

Datasets. All experiments use the publicly available Refactory dataset (Hu et al. 2019); our seeded, scripted import pipeline (test parsing, per-test entry points, split sampling) is released, so the exact evaluation sets are reconstructible from the public corpus. No new human-subject data are collected.

Computational experiments. Source code, prompts, synthesized repairers, raw LLM responses, the ported Refactory-tool configuration, and analysis scripts are released; experiments run on CPU only. Model versions, pricing snapshot, tokenizer convention, decoding budget (single sample), split seed, timeouts, and the RPS threshold are stated in Experimental Setup and fixed in the released configuration. Metrics are formally defined; repair rates carry 95% Wilson intervals and paired claims use exact McNemar tests. Local search and evaluation are deterministic; per-submission LLM calls are single-shot by design (deployment matches one attempt per submission). Builder synthesis is stochastic; every round's prompt, response, and build-set outcome for both builder tiers is released, and the refinement loop's convergence behavior is reported in RQ3.